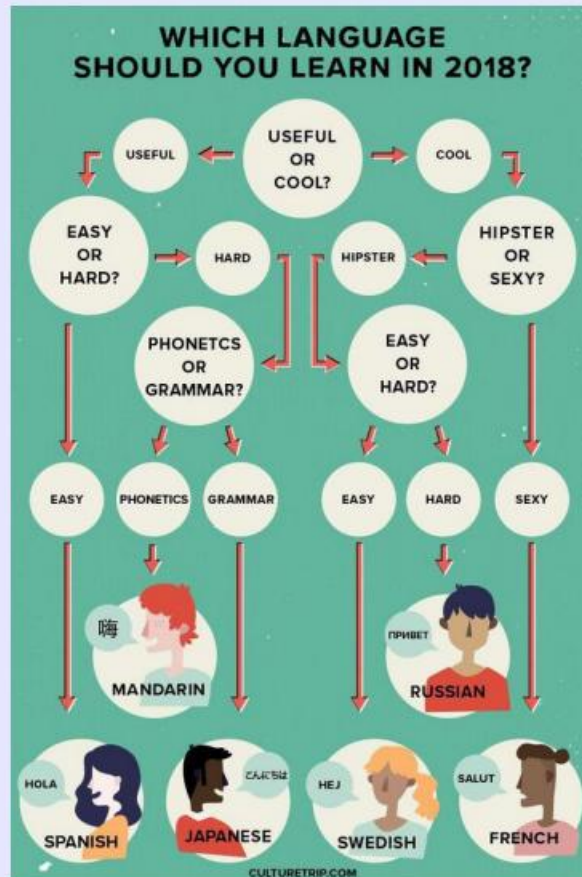
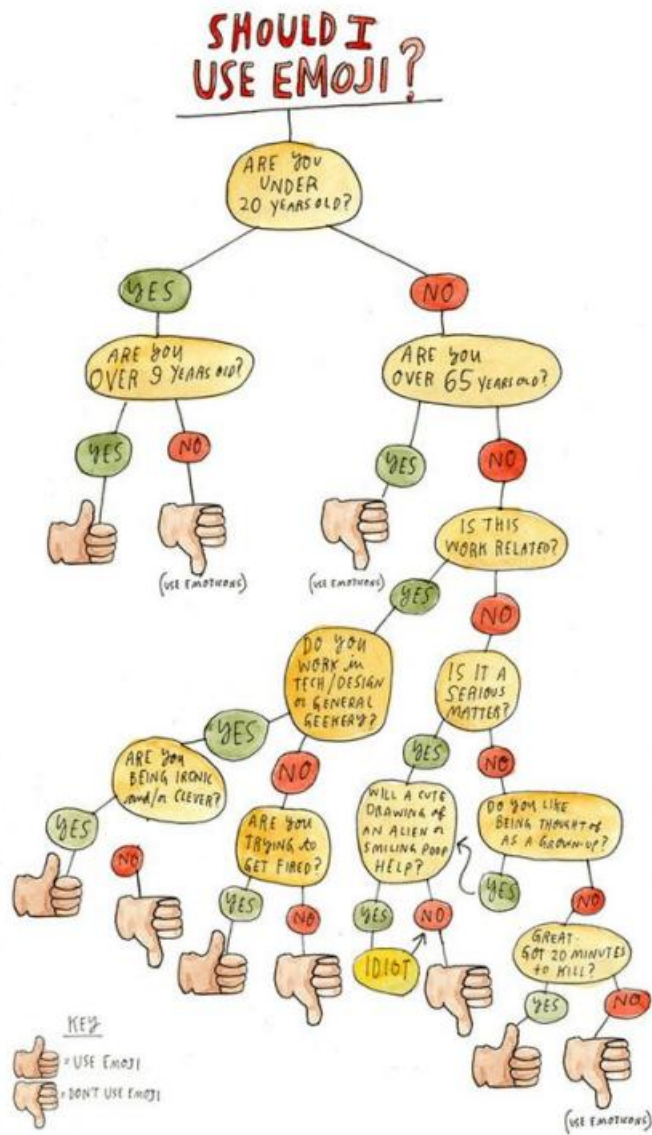


Our first machine learning algorithm will be decision trees. A decision tree is a very common algorithm that we humans use to make many different decisions. You may be using one without realizing it. Here are some examples of decision trees.

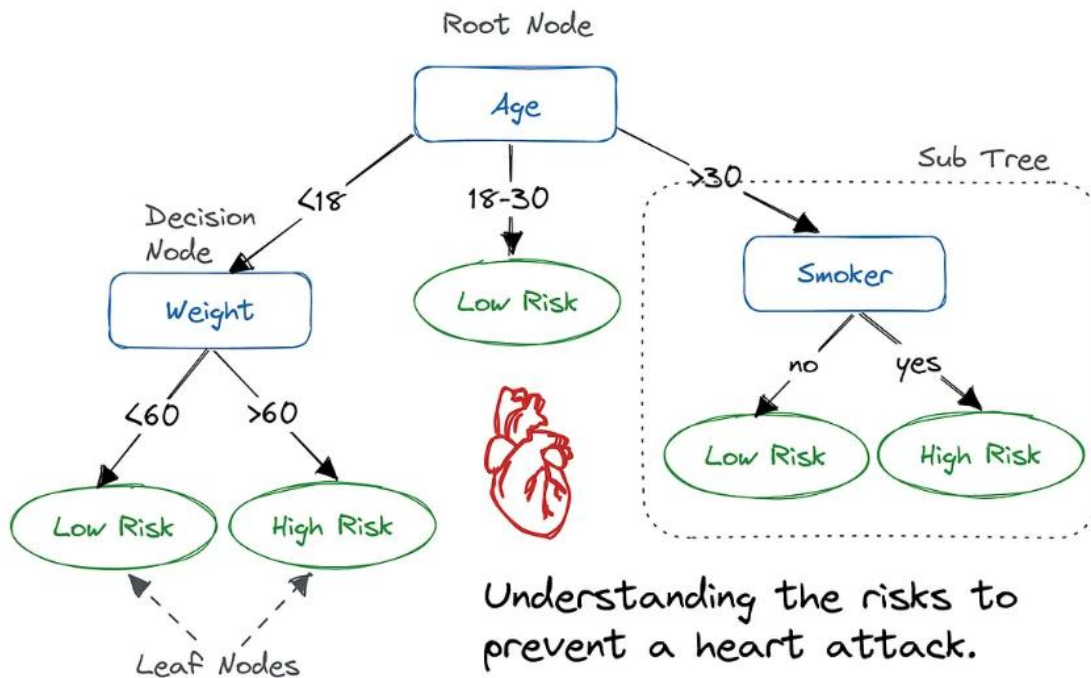
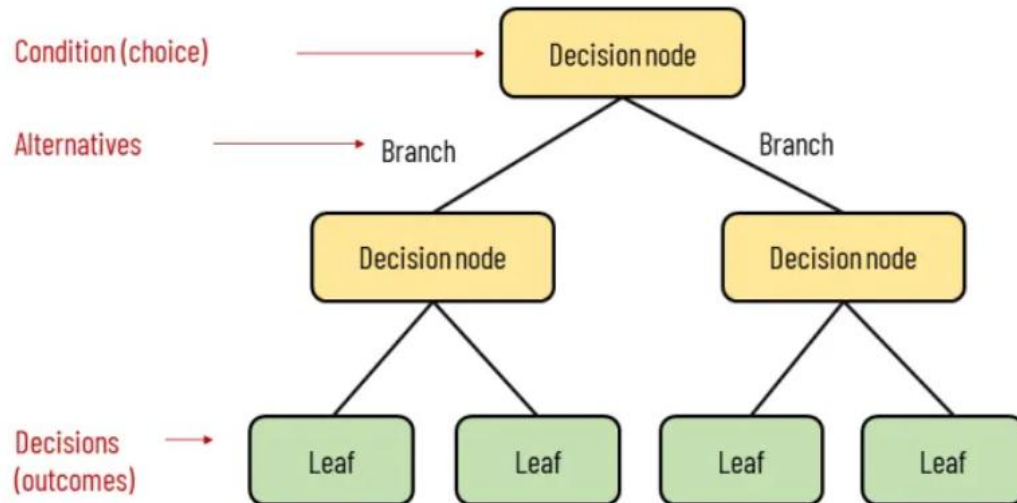
Example: Which language should you learn?



Example: Should you use emoji in a conversation?



Elements of a decision tree



Type

- **Classification Trees:** Used when the target variable is categorical.
- **Regression Trees:** Used when the target variable is continuous.

Mathematics

The core of a decision tree lies in partitioning the dataset using splitting criteria:

Gini Impurity:

Gini Impurity is a measure of how often a randomly chosen element would be incorrectly classified. It ranges between 0 (perfectly classified) and 0.5 (worst classification).

Mathematical Formula for Gini Impurity:

$$G = 1 - \sum_{i=1}^C p_i^2$$

Intuition:

- **Gini = 0** means the node is pure, i.e., all instances belong to the same class.

- The more diverse the classes are in a node, the higher the Gini Impurity.

Entropy:

Entropy measures the amount of disorder or uncertainty in the data. The higher the entropy, the more mixed or uncertain the data is, and the lower it is, the purer the data.

Mathematical Formula for Entropy:

$$H = - \sum_{i=1}^C p_i \log_2(p_i)$$

= probability

Intuition:

- **Entropy = 0** means the node is pure (no uncertainty).
- The more mixed the classes are in a node, the higher the entropy.

Mathematical Equation

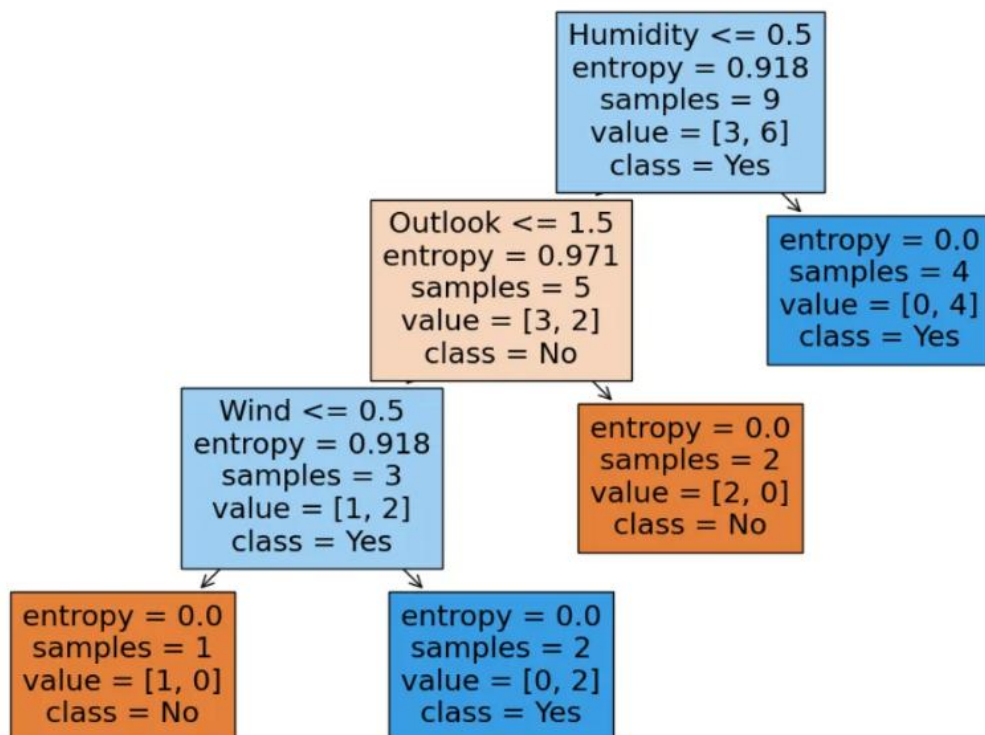
- For regression trees:

$$y = \frac{1}{N} \sum_{i=1}^N y_i$$

- For classification trees, it uses the probabilities for each class.

Comparing Gini Impurity and Entropy

Both Gini and Entropy help decide the best splits in a decision tree. Lower values are preferable as they indicate higher purity. Gini tends to be computationally faster, while Entropy can be more informative, especially when there's a mix of class distributions.



Steps:

Step 1: Load and Preprocess the Data Before any calculations happen, the raw text and email features are converted into structured numbers).

Target Label (y): The output to predict (e.g., 1 for Spam, 0 for Not Spam).

Features (X): Attributes like `has_word_win` (1/0), `num_links` (integer), or `caps_ratio` (float).

Step 2: Calculate Total Impurity of the Entire Dataset (Parent Node)

The algorithm first looks at the entire dataset as one big pool and calculates how mixed (impure) it is using **Entropy** or **Gini Impurity**.

If your loaded dataset has N total emails, it calculates:

$$\text{Entropy}(S) = -p_{\text{spam}} \log_2(p_{\text{spam}}) - p_{\text{not_spam}} \log_2(p_{\text{not_spam}})$$

- p_{spam} : Fraction of emails that are spam ($\frac{\text{Spam Count}}{\text{Total Count}}$).
- $p_{\text{not_spam}}$: Fraction of emails that are safe.

..... **Goal:** The algorithm wants to split the dataset so that the sub-groups become as pure as possible (Entropy close to 0).

Step 3: Evaluate Every Possible Split Condition

The algorithm tests **every single feature** (and every possible threshold for that feature) to see what happens if it splits the data based on a specific question.

For each potential split (e.g., *"Does the email contain 'WIN'?"*):

1. It creates two sub-groups: **Left Branch (YES)** and **Right Branch (NO)**.
2. It calculates the **Entropy** for the Left Branch ($\text{Entropy}_{\text{left}}$).
3. It calculates the **Entropy** for the Right Branch ($\text{Entropy}_{\text{right}}$).
4. It combines them into a weighted average:

$$\text{Weighted Entropy} = \left(\frac{N_{\text{left}}}{N_{\text{total}}} \right) \text{Entropy}_{\text{left}} + \left(\frac{N_{\text{right}}}{N_{\text{total}}} \right) \text{Entropy}_{\text{right}}$$

Step 4: Calculate Information Gain for All Features

To determine which feature is the most useful split, the algorithm calculates **Information Gain** for each feature test:

$$\text{Information Gain} = \text{Parent Entropy} - \text{Weighted Entropy of Split}$$

- **High Information Gain:** The split effectively separates Spam from Not Spam (great test).
- **Low Information Gain:** The split leaves both branches just as mixed as before (useless test).

Step 5: Select the Best Feature as the Node

The algorithm compares the Information Gain values calculated across all candidate features:

1. The feature with the **highest Information Gain** wins.
2. That feature becomes the decision question for the current node (e.g., Root Node).
3. The dataset is physically split into two child subsets based on that rule.

Step 6: Repeat Recursively (Tree Growth)

The algorithm repeats **Steps 2 through 5** independently on each child subset:

- It checks the left branch, evaluates all remaining features, and finds the next best split.
- It checks the right branch and does the same.

This recursive process continues down the tree branches.

Step 7: Stopping Criteria (Creating Leaf Nodes)

The algorithm stops splitting a branch and creates a final classification label (**Leaf Node**) when one of the following conditions is met:

- **Pure Subset:** All emails in that branch are 100% Spam or 100% Not Spam (Entropy = 0).
- **Max Depth Reached:** The tree has reached a user-defined limit on depth (e.g., max 4 levels deep to prevent overfitting).
- **Minimum Samples:** The branch has fewer samples than a set threshold (e.g., fewer than 5 emails left).

Once stopped, the leaf node is assigned the majority class of the emails in that bucket.